

WEEK 8 — Unit 3: Microservices with Docker Compose

Topic: Docker Compose – YAML Structure, Services, Volumes, Networks & Environment Variables
Course: INT 332 – DevOps

Note 1: Introduction to Docker Compose and YAML Structure

What is Docker Compose?

Docker Compose is a tool that allows you to define and run multi-container Docker applications. Instead of starting each container manually with long docker run commands, you describe your entire application stack in a single file called docker-compose.yml, and then bring the whole stack up or down with a single command.

Docker Compose is particularly useful in microservices environments where multiple containers (database, backend, frontend, cache, etc.) need to work together and must be configured to communicate with each other.

Why Docker Compose?

- Running multiple docker run commands manually is error-prone and hard to reproduce
- Compose allows the entire application environment to be version-controlled alongside the code
- Makes it easy to share a working environment with teammates — anyone can run docker compose up and get an identical setup
- Handles container networking, volume mounting, and environment configuration in one place

The docker-compose.yml File

The core of Docker Compose is the docker-compose.yml file. It is written in YAML (YAML Ain't Markup Language), a human-readable data serialisation format that uses indentation to represent structure.

YAML Basics You Must Know

- YAML uses spaces for indentation — never tabs
- Two spaces per level is the standard convention
- Key-value pairs are written as key: value
- Lists are written with a dash and space before each item (- item)

- Strings do not always need quotes, but quotes are recommended when values contain special characters
- Comments are written with #

Example of basic YAML syntax:

name: my-app # string value

port: 8080 # integer value

enabled: true # boolean value

tags:

- web

- backend

- api

Top-Level Structure of docker-compose.yml

A typical docker-compose.yml file has the following top-level keys:

version: "3.9"

services:

 service-name:

 ...

volumes:

 volume-name:

networks:

 network-name:

Each of these top-level sections is described in detail below.

The version Field

The version field specifies which version of the Docker Compose file format you are using. Different versions support different features.

version: "3.9"

- Version 3.x is the most commonly used and is compatible with Docker Swarm
- As of recent Docker Compose V2 releases, the version field is considered optional and may be omitted, but it is still good practice to include it for clarity
- Version 2.x was specific to single-host deployments and is now largely legacy

The services Section

The services section is the most important part of the file. Each service represents one container. You define the container's image, ports, volumes, environment variables, and dependencies here.

services:

web:

image: nginx:latest

ports:

- "80:80"

app:

build: ./app

ports:

- "3000:3000"

db:

image: postgres:15

environment:

POSTGRES_PASSWORD: secret

Each key under services is the name you give to that container within the Compose network. These names also act as hostnames — so the app container can reach the db container simply by using db as the hostname.
